Vivado（含可用的Batch模式） 2022.2 on Apple Silicon Mac（不完善）

注：笔者能力十十十十十分有限，只能写到这个程度了。。。如果以后能力有提升会完善这篇文章的。

> 本文的部署流程是通过AI辅助完成的，总结这篇文章也是为了记录在 linux 系统里的“摸爬滚打”。这篇文章**没有任何技术含量**，只是记录我装机折腾的种种过程。毕竟跟谁学都是学，包括AI。现在列出步骤只是为了说明该方案的可行性及便捷性。笔者刚刚接触 linux，是小白，实在不熟悉这个系统的环境和操作。步骤只是大致的框架，如遇到任何问题，请做“Prompt工程师”。

详见：github copilot 学生认证。https://github.com/education/students

在 Ubuntu 中打开 VSCode 的 Agent ，直接让它帮你来（hhh）。

- 写作工具：Typora；环境：Ubuntu 24.04 ARM64；网页构建工具：Mkdocs。

1. 目标与版本

目标是在 **Apple Silicon Mac + Parallels Desktop + Ubuntu 24.04 ARM64** 上运行 **Vivado 2022.2**。

- 宿主机：macOS（Apple Silicon）
- 虚拟化：Parallels Desktop 26
- 虚拟机系统：Ubuntu 24.04 with Rosetta
- Vivado 版本：**2022.2**

2. 需要下载的内容

2.1 Parallels Desktop

- **这个步骤可能是文章唯一给出的有用信息hhh**。

  废话不多说，上链接：https://macwk.cn/app/2941.html。

  > ⚠ **Warning**
  >
  > **叠甲** 仅供学习使用！请在24小时之内删除。请支持正版软件。

  如果签名失败，请先将安装容器的所有文件复制并粘贴到一个新建文件夹中，然后执行：

  ```
  xattr -cr "你的文件名"    # 可以通过拖拽的方式添加，不要直接添加安装包中的文件。
  ```

  Fence 1

  可能需要在[设置] - [隐私与安全性]中按指纹解锁权限继续安装。

2.2 Ubuntu 24.04 ARM64 镜像（在 Parallels 中创建虚拟机）

点击新建选项，选择`下载 Ubuntu with Rosetta`，等待自动下载完毕。该软件完成度很高，几乎不用手动操作。

Image 1

2.3 Vivado 2022.2 Linux 安装包

常见文件名示例（以官方实际页面为准）：

- `Xilinx_Unified_2022.2_1014_8888_Lin64.bin`

具体下载方式请见课程文档。

3. 执行指令并部署

> **Note**
>
> 这些指令只是根据此前聊天记录中试错后有效的指令，并未在全新的环境中测试。如有卡壳，请找 Agent。

3.0.1 第 0 步：Xorg 会话背景

这一步的目的：解释为什么文中后续会强调 `x11/xorg`。

- Ubuntu 24.04 桌面默认更偏向 Wayland，但 Vivado 2022.2 的 GUI 组件对 Wayland 兼容并不稳定。
- 在 Wayland 下常见现象是：启动后闪退、窗口渲染异常、焦点/输入异常，或者偶发段错误。
- `GDK_BACKEND=x11` 只能告诉 GTK 尽量走 X11 后端，但它**不能替代登录级别的会话切换**。
- 所以Vivado GUI 场景尽量在 **Xorg 会话**（`XDG_SESSION_TYPE=x11`） 下运行。

快速判断当前会话：

```
echo "$XDG_SESSION_TYPE"
```

Fence 2

输出为 `x11` 说明已在 Xorg 会话；输出为 `wayland` 则建议切换（登出并重新登陆时右下角的小齿轮中选择）后再做 GUI 相关步骤。

3.1 第 1 步：安装 Ubuntu 基础依赖

这一步的目的：让系统具备“识别并接管 x86_64 程序”的基础能力。

```
sudo apt update
sudo apt install -y binfmt-support qemu-user-static
```

Fence 3

3.2 第 2 步：配置 RosettaLinux 的 binfmt 路由

这条指令在mac中的终端执行，其余均为linux终端指令：

```
softwareupdate --install-rosetta --agree-to-license
```

Fence 4

这一步的目的：告诉 Ubuntu “遇到 x86_64 程序时该交给谁来运行”。

先检查：

```
ls /proc/sys/fs/binfmt_misc
cat /proc/sys/fs/binfmt_misc/RosettaLinux
```

Fence 5

如果 `x86_64` 被 box64 接管，先禁用：

```
echo -1 | sudo tee /proc/sys/fs/binfmt_misc/x86_64
```

Fence 6

启用 RosettaLinux：

```
sudo update-binfmts --enable RosettaLinux
```

Fence 7

持久化（重启后仍生效）：

```
sudo install -d /etc/binfmt.d

cat <<'EOF' | sudo tee /etc/binfmt.d/rosetta-linux.conf >/dev/null
:RosettaLinux:M::\x7fELF\x02\x01\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x02\x00\x3e\x00:\xff\xff\xff\xff\xff\xfe\xfe\x00\xff\xff\xff\xff\xff\xff\xff\xff\xfe\xff\xff\xff:/media/psf/RosettaLinux/rosetta:OCF
EOF

cat <<'EOF' | sudo tee /etc/binfmt.d/x86_64-rosetta.conf >/dev/null
:x86_64:M::\x7fELF\x02\x01\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x02\x00\x3e\x00::/media/psf/RosettaLinux/rosetta:OCF
EOF

sudo systemctl restart systemd-binfmt
```

下面指令解决 psf 可能晚挂载导致开机失效的问题。（笔者的Rosetta在挂载目录中）

```
cat <<'EOF' | sudo tee /usr/local/sbin/refresh-rosetta-binfmt >/dev/null
#!/usr/bin/env bash
set -euo pipefail
for i in $(seq 1 120); do
  if [ -x /media/psf/RosettaLinux/rosetta ]; then
    systemctl restart systemd-binfmt
    exit 0
  fi
  sleep 1
done
echo "Rosetta interpreter not found: /media/psf/RosettaLinux/rosetta" >&2
exit 1
EOF

sudo chmod +x /usr/local/sbin/refresh-rosetta-binfmt

cat <<'EOF' | sudo tee /etc/systemd/system/refresh-rosetta-binfmt.service >/dev/null
[Unit]
Description=Refresh Rosetta binfmt after shared folders are ready
After=multi-user.target

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/refresh-rosetta-binfmt

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now refresh-rosetta-binfmt.service
```

Fence 9

> 如果报错且您没有头绪，请咨询Agent。

3.3 第 3 步：安装 Vivado 所需 amd64 运行库

这一步的目的：补齐 Vivado 自带 x86_64 JRE 需要的运行库，避免启动时报错。

```
sudo dpkg --add-architecture amd64
sudo apt update
sudo apt install -y zlib1g:amd64 libstdc++6:amd64 libc6:amd64
```

Fence 10

3.4 第 4 步：下载并解包 Vivado 2022.2

这一步的目的：拿到 `xsetup` 安装器文件。

```
chmod +x Xilinx_Unified_2022.2_1014_8888_Lin64.bin
./Xilinx_Unified_2022.2_1014_8888_Lin64.bin --target ~/xilinx_2022.2
```

Fence 11

解包完成后，`xsetup` 路径通常是：`~/xilinx_2022.2/xsetup`。

3.5 第 5 步：修改 `xsetup` 架构检查

只改“报错退出”的架构判断分支，其他安装逻辑不要动。仅需把exit1前面加`#`注释掉即可。图片中是AI修改后的片段，不必这么麻烦。

Image 2

3.6 第 6 步：运行安装器

这一步的目的：正式安装 Vivado 到系统目录。

```
cd ~/xilinx_2022.2
./xsetup
```

Fence 12

按默认路径，推荐安装到 `/opt/Xilinx`，与仓库中Makefile的写法一致。

3.7 第 6.5 步：切换到 Xorg 会话（GUI 相关步骤前）

这一步的目的：让后续 GUI 启动脚本配置在更稳定的显示栈上工作。

1. 先退出登录（不是重启也可以）。
2. 在登录界面选中你的用户后，点击右下角齿轮图标。
3. 选择 **Ubuntu on Xorg**（或带 Xorg 字样的会话）。
4. 登录后执行：

```
echo "$XDG_SESSION_TYPE"
```

Fence 13

若输出为 `x11`，再继续第 7 步。

3.8 第 7 步：安装好后，修改官方 GUI 启动脚本

这一步的目的：把 GUI 兼容参数收敛到官方入口。

先备份：

```
sudo cp /opt/Xilinx/Vivado/2022.2/bin/vivado /opt/Xilinx/Vivado/2022.2/bin/vivado.bak
```

Fence 14

编辑文件 `/opt/Xilinx/Vivado/2022.2/bin/vivado`，下面是AI修改后的启动脚本。务必做好备份，不要盲目修改。建议让AI接管。

```
#!/bin/bash
. "`dirname \"$0\"`/setupEnv.sh"
# Set XILINX_VIVADO
XILINX_VIVADO=`dirname "$RDI_BINROOT"`
export XILINX_VIVADO

export RDI_DEPENDENCY="XILINX_VIVADO_HLS:XILINX_HLS"
export _RDI_NEEDS_PYTHON=True
export RDI_USE_JDK11=True

RDI_LAUNCH_MODE="gui"
while [ $# -gt 0 ]; do
    if [ "$1" = "-mode" ] && [ $# -ge 2 ]; then
        RDI_LAUNCH_MODE="$2"
        break
    fi
    shift
done

if [ -n "${HOME:-}" ]; then
    mkdir -p "$HOME/.Xilinx/Vivado/2022.2/data/layouts" 2>/dev/null || true
fi

export GDK_BACKEND=${GDK_BACKEND:-x11}
export SWT_GTK3=${SWT_GTK3:-0}
export LIBGL_ALWAYS_SOFTWARE=${LIBGL_ALWAYS_SOFTWARE:-1}
if [ "${XDG_SESSION_TYPE:-}" = "x11" ]; then
    export _JAVA_AWT_WM_NONREPARENTING=0
else
    export _JAVA_AWT_WM_NONREPARENTING=1
fi
export XLNX_USE_NATIVE_DECORATION=${XLNX_USE_NATIVE_DECORATION:-1}
if [ "$RDI_LAUNCH_MODE" = "gui" ]; then
    if [ -n "${HOME:-}" ]; then
        rm -f "$HOME/.Xilinx/Vivado/2022.2/data/layouts/application/default.layout" 2>/dev/null || true
```

```
    fi
    export _VIVADO_STABLE_JAVA_OPTS="-Xint -XX:+UseSerialGC -Dsun.java2d.xrender=false -Dsun.java2d.opengl=false -Dsun.awt.disablegrab=true -Dsun.java2d.uiScale=1.0 -Dawt.useSystemAAFontSettings=on"
    export _JAVA_OPTIONS="$_VIVADO_STABLE_JAVA_OPTS"
    export JAVA_TOOL_OPTIONS="$_VIVADO_STABLE_JAVA_OPTS"
    export JDK_JAVA_OPTIONS="$_VIVADO_STABLE_JAVA_OPTS"
else
    export _VIVADO_STABLE_JAVA_OPTS="-Dsun.java2d.xrender=false -Dsun.awt.disablegrab=true"
    [ -z "${_JAVA_OPTIONS:-}" ] && export _JAVA_OPTIONS="$_VIVADO_STABLE_JAVA_OPTS"
    [ -z "${JAVA_TOOL_OPTIONS:-}" ] && export JAVA_TOOL_OPTIONS="$_VIVADO_STABLE_JAVA_OPTS"
    [ -z "${JDK_JAVA_OPTIONS:-}" ] && export JDK_JAVA_OPTIONS="$_VIVADO_STABLE_JAVA_OPTS"
fi

# CR-1074520: Disable bmalloc memory allocator and use the system default one
#   instead. This prevents WebKit from overindulging in virtual memory on high
#   memory machines. https://trac.webkit.org/wiki/EnvironmentVariables
export Malloc=1
##
# Launch the loader and specify the executable to launch
##
#
# Loader arguments:
#   -exec   -- Name of executable to launch
##
RDI_PROG=`basename "$0"`
"$RDI_BINROOT"/loader -exec $RDI_PROG "$@"
```

Fence 15

> 不要把这两个变量写到 `/etc/profile` 或 `/etc/profile.d`。

3.9 第 8 步：验证（Batch）

打开实验仓库，修改Makefile，执行`make bitstream`，如果成功生成二进制文件，则完成。否则，如果没有头绪，请继续让Agent帮忙调试。（实在是无奈，我们本**不该过度依赖AI**的...奈何刚接触这个环境...）

3.10 第 9 步：GUI 启动（统一入口）

点击桌面图标，查看是否进入程序。点击任意项目，生成波形。

建议先确认会话类型：

```
echo "$XDG_SESSION_TYPE"
```

Fence 16

如不是 `x11`，优先回到第 6.5 步切换 Xorg 后再测 GUI。

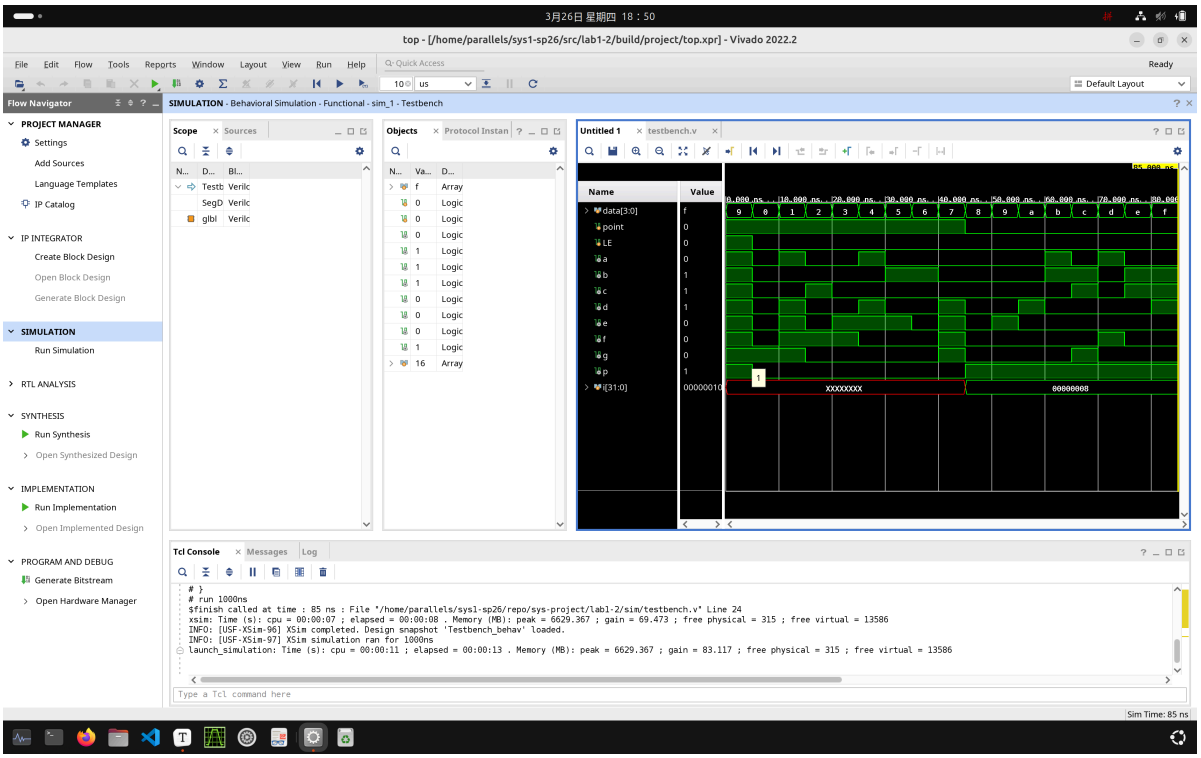

Image 3

4. 复现检查

分别登出和重启后重新尝试验证。